Final Project: Bike Sharing Demand Prediction

Introduction to Data Analysis and Machine Learning with Python

name:liuyoulin studentID:2025141220316

Contents

1. Introduction
2. Dataset Overview
3. Exploratory Data Analysis
4. Data Preprocessing
5. Regression Models
6. Initial Performance Evaluation
7. Outlier Detection and Removal
8. Hyperparameter Optimization
9. Final Results and Comparison
10. Discussion and Conclusion

1. Introduction

This project compares two different regression algorithms on a real-world bike sharing dataset. The goal is to predict the total number of bike rentals per hour (denoted as `cnt`) based on various temporal, seasonal, and weather-related features. Two regressors are selected for comparison: **Random Forest Regressor** and **K-Nearest Neighbors (KNN) Regressor**.

The analysis follows these steps: (1) data exploration and visualization, (2) data preprocessing, (3) initial model evaluation with default parameters, (4) outlier detection and removal, (5) hyperparameter tuning, and (6) final comparison of model performance based on training time and mean squared error (MSE) on both training and test sets.

2. Dataset Overview

The dataset contains hourly bike rental data spanning two years (2011 and 2012), with 17,379 total observations. The target variable is `cnt`, representing the total number of bike rentals in a given hour. Features include time/seasonal information and weather measurements.

2.1 Feature Description

Feature	Type	Description
season	Categorical	Season (1=spring, 2=summer, 3=fall, 4=winter)
yr	Categorical	Year (0=2011, 1=2012)
mnth	Categorical	Month (1 to 12)
hr	Categorical	Hour of day (0 to 23)
holiday	Categorical	Whether day is a holiday (0=no, 1=yes)
weekday	Categorical	Day of the week (0 to 6)
workingday	Categorical	Whether day is a working day (0=no, 1=yes)
weathersit	Categorical	Weather situation (1=clear to 4=severe)
temp	Numerical	Normalized temperature
atemp	Numerical	Normalized "feels like" temperature
hum	Numerical	Normalized humidity
windspeed	Numerical	Normalized wind speed
cnt	Numerical	**Total bike rentals (target variable)**

2.2 Basic Statistics

The target variable `cnt` ranges from 1 to 977 rentals per hour. The mean is approximately 189 and the median is 142, indicating a right-skewed distribution typical for count data.

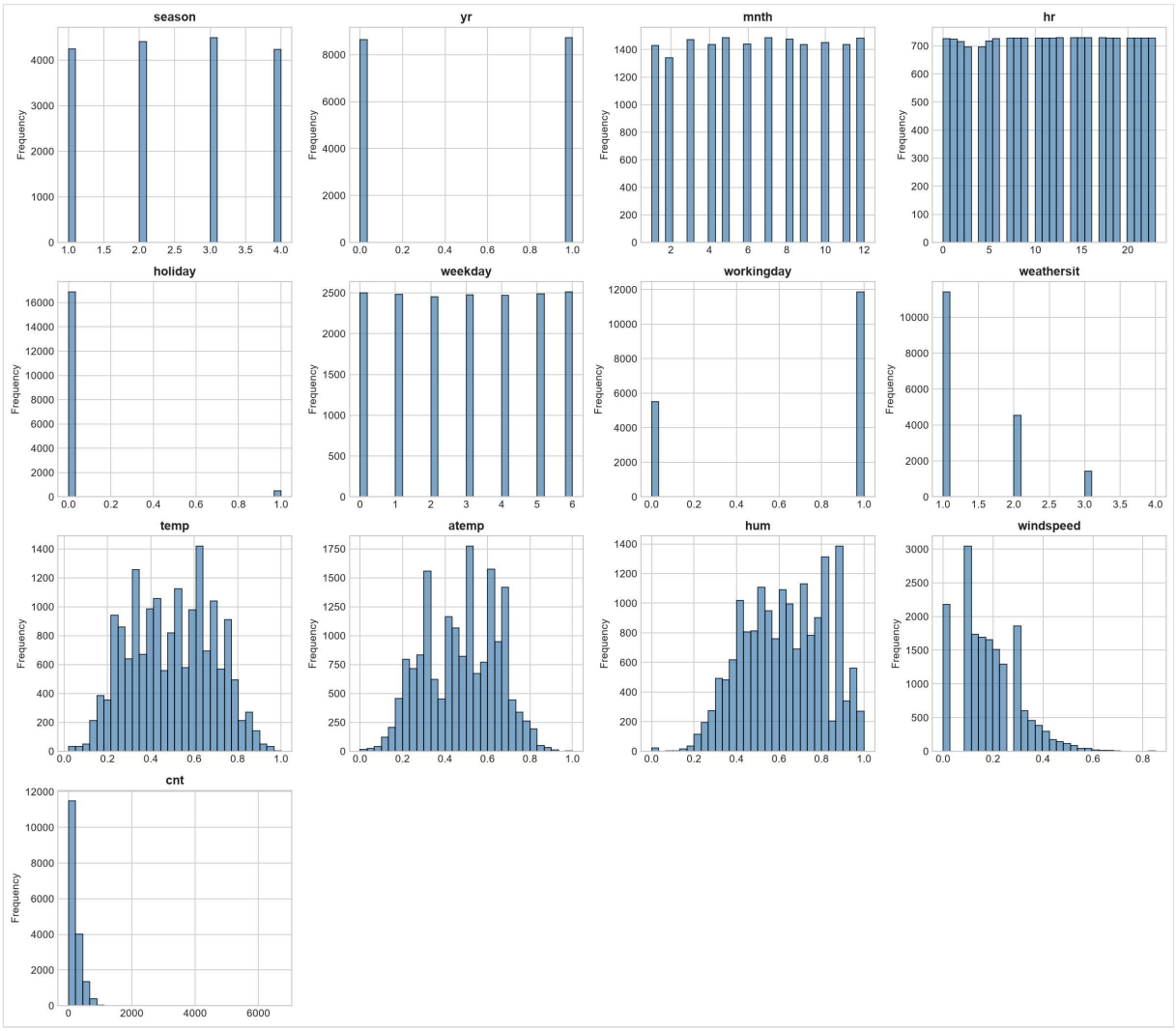


Figure 1. Distribution histograms for all 13 variables in the dataset.

3. Exploratory Data Analysis

3.1 Target Variable Distribution

The target variable `cnt` shows a clear right-skewed distribution. Most hours have relatively low rental counts (below 200), but there is a long tail extending up to nearly 1000 rentals during peak hours. The mean (189) is noticeably higher than the median (142), confirming the positive skew.

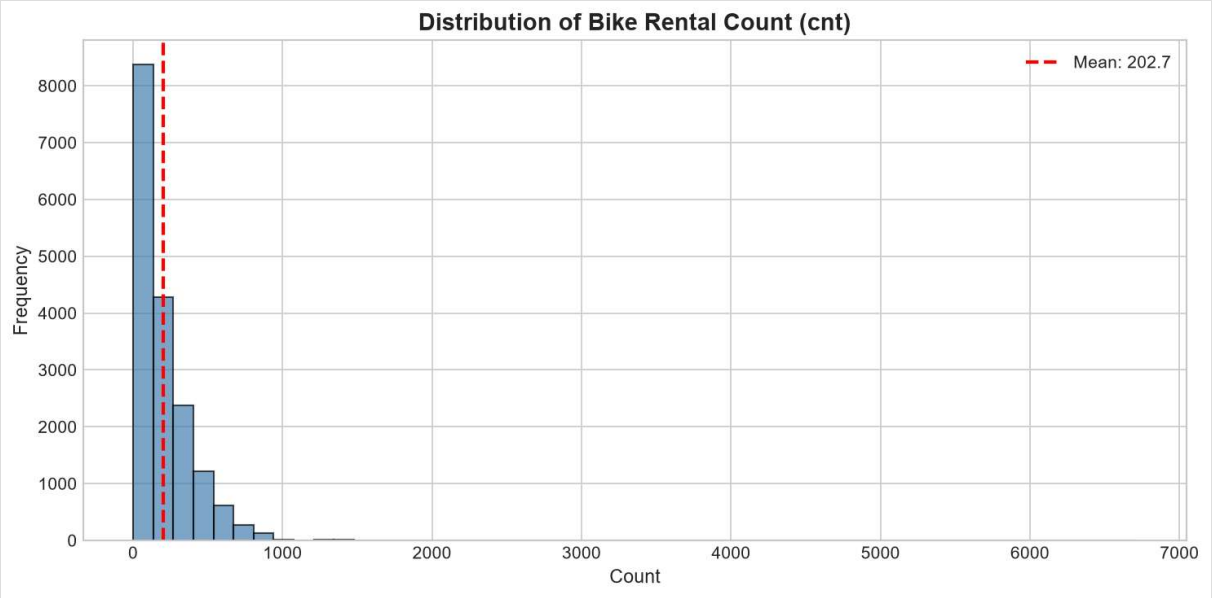


Figure 2. Distribution of bike rental counts with mean indicated by the dashed red line.

3.2 Hourly Pattern Analysis

One of the strongest patterns emerges when examining average rentals by hour of day. There are clear peak periods around 8 AM (morning commute) and 5-6 PM (evening commute), with a midday lull. This bimodal pattern reflects typical workday commuting behavior and suggests that the `hr` feature will be one of the most important predictors.

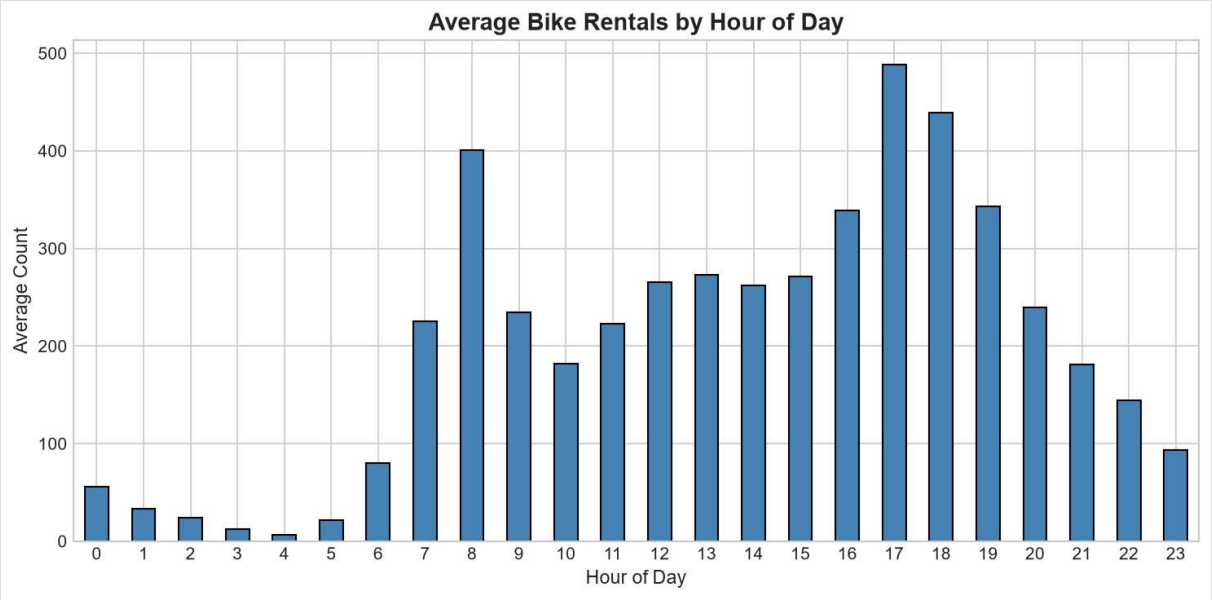


Figure 3. Average bike rentals by hour of day, showing clear morning and evening commuting peaks.

3.3 Correlation Analysis

The correlation matrix reveals several expected relationships. Temperature (`temp`) and "feels like" temperature (`atemp`) are highly correlated (0.98), which makes sense as they measure similar quantities. Both temperature variables show a moderate positive correlation with rental count. Humidity has a weak negative correlation, while wind speed shows a small positive correlation.

Among categorical variables, year (`yr`) has a notable positive correlation, indicating growth in bike rentals from 2011 to 2012. Weather situation shows a negative correlation, as expected — worse weather reduces rentals.

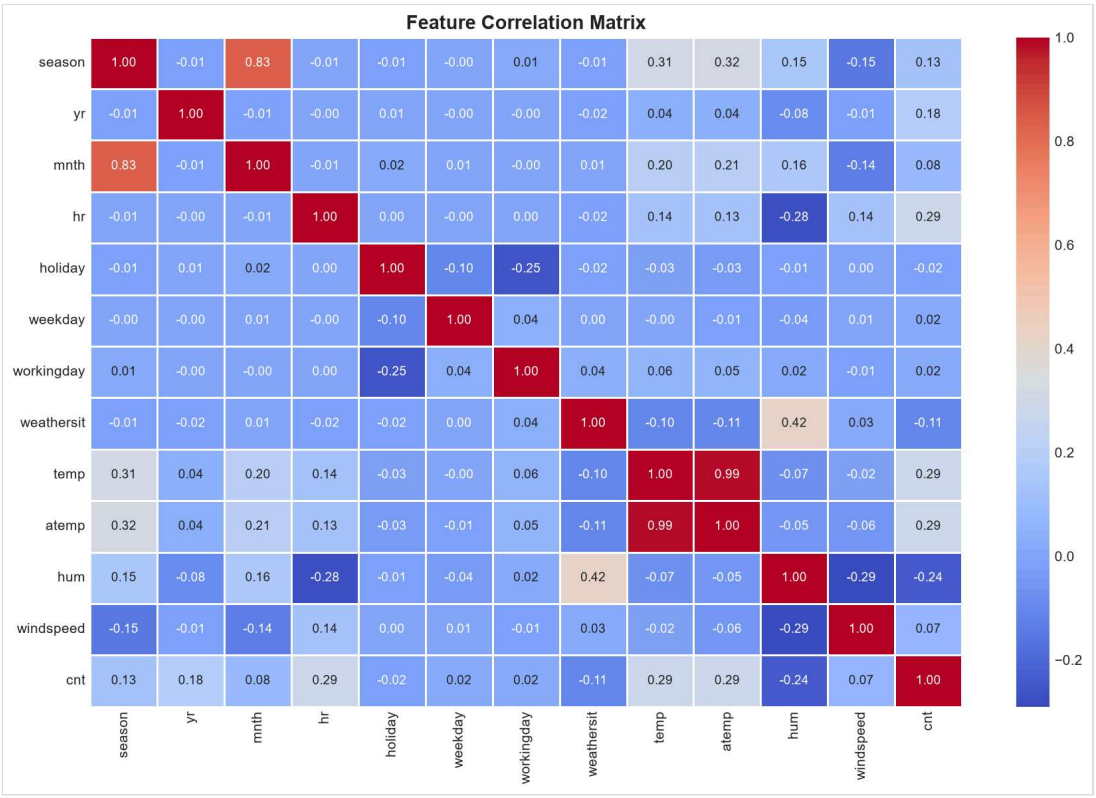


Figure 4. Pearson correlation heatmap for all features and the target variable.

4. Data Preprocessing

4.1 Categorical Encoding

Features `season`, `yr`, `mnth`, `hr`, `holiday`, `weekday`, `workingday`, and `weathersit` are treated as categorical variables despite being stored as integers. We apply one-hot encoding (dummy encoding) to convert these into binary columns, dropping the first level to avoid the dummy variable trap.

After one-hot encoding, the feature space expands from 12 original features to 57 features, due to the high cardinality of `hr` (24 values) and `mnth` (12 values).

4.2 Train-Test Split

The data is randomly split into an 80% training set and a 20% test set using a fixed random state (seed=42) for reproducibility.

4.3 Feature Scaling

Since KNN is a distance-based algorithm, feature scaling is critical for fair distance computation. We apply standard scaling (zero mean, unit variance) to all features for the KNN model. Random Forest is not sensitive to feature scales, so the raw encoded features are used directly.

```
# One-hot encode categorical variables
X_encoded = pd.get_dummies(X, columns=categorical_cols, drop_first=True)

# 80/20 train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X_encoded, y, test_size=0.2, random_state=42
)

# Standard scaling for KNN only
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

5. Regression Models

5.1 Random Forest Regressor

Random Forest is an ensemble learning method that builds multiple decision trees during training and outputs the average prediction of the individual trees. It is well-suited for this problem because it can capture non-linear relationships and feature interactions — for example, how the effect of temperature varies by season, or how hourly patterns differ between workdays and weekends. It also provides feature importance scores for interpretability.

5.2 K-Nearest Neighbors Regressor

KNN is an instance-based (lazy) learning algorithm that predicts the target for a new point by averaging the target values of its k nearest neighbors in the feature space. It is conceptually simple but can struggle with high-dimensional data due to the curse of dimensionality, where all points become roughly equidistant. We include it as a baseline comparison against the more powerful Random Forest.

6. Initial Performance Evaluation

We first train both models with their default hyperparameters on the full dataset (before outlier removal) to establish baseline performance.

6.1 Default Parameters

- **Random Forest:** n_estimators=100, max_depth=None (unlimited depth)
- **KNN:** n_neighbors=5

6.2 Initial Results

Metric	Random Forest (Default)	KNN (Default, k=5)
Training Time (seconds)	0.6487	0.0025
Train MSE	6,484.15	32,216.49
Test MSE	39,552.47	44,593.08

6.3 Initial Predicted vs Actual Plots

The scatter plots of predicted versus actual values reveal that both models have difficulty with high-count data points. There is a noticeable spread of points away from the diagonal at higher rental counts, with many actual values far exceeding the predictions. This pattern suggests the presence of outlier data points that the models cannot fit well.

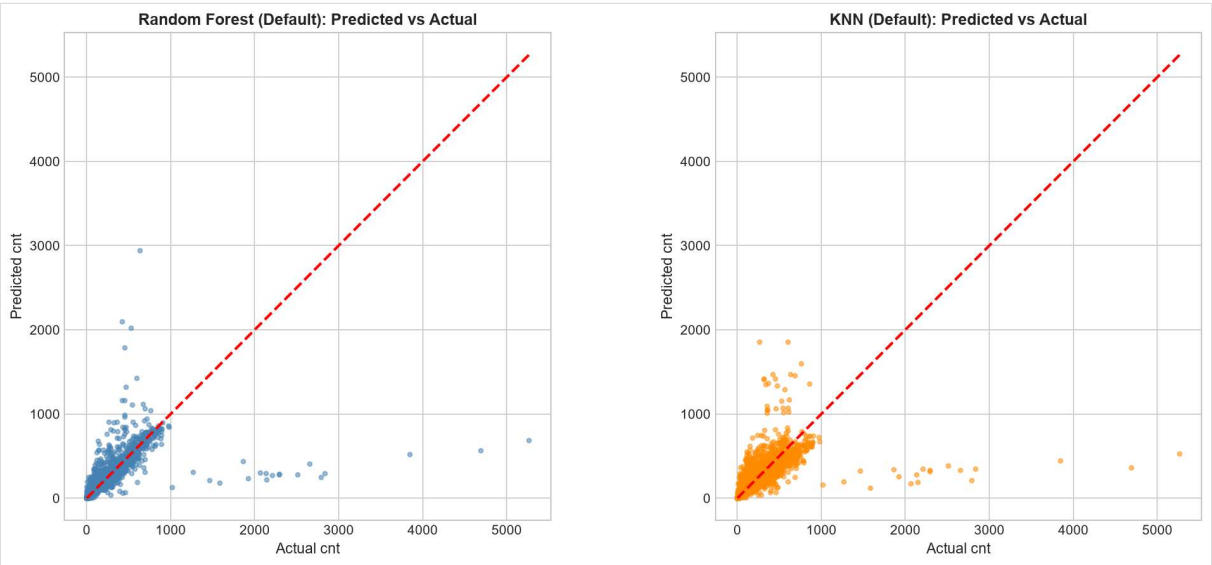


Figure 5. Predicted vs actual rental counts with default parameters. Left: Random Forest. Right: KNN.

7. Outlier Detection and Removal

7.1 Identifying Outliers

We use the Interquartile Range (IQR) method to detect outliers in the target variable `cnt`. The IQR is the range between the 25th percentile (Q1) and 75th percentile (Q3). Any data point falling below Q1 − 1.5×IQR or above Q3 + 1.5×IQR is classified as an outlier.

Statistic	Value
Q1 (25th percentile)	40.0
Q3 (75th percentile)	284.5
IQR	244.5
Lower bound (Q1 − 1.5×IQR)	−326.8
Upper bound (Q3 + 1.5×IQR)	651.2

Since rental count cannot be negative, the lower bound is not meaningful. Using the upper bound of 651.2, we identify 577 outlier samples (approximately 3.3% of the data). These are hours with exceptionally high rental demand that may be driven by special events not captured by our features.

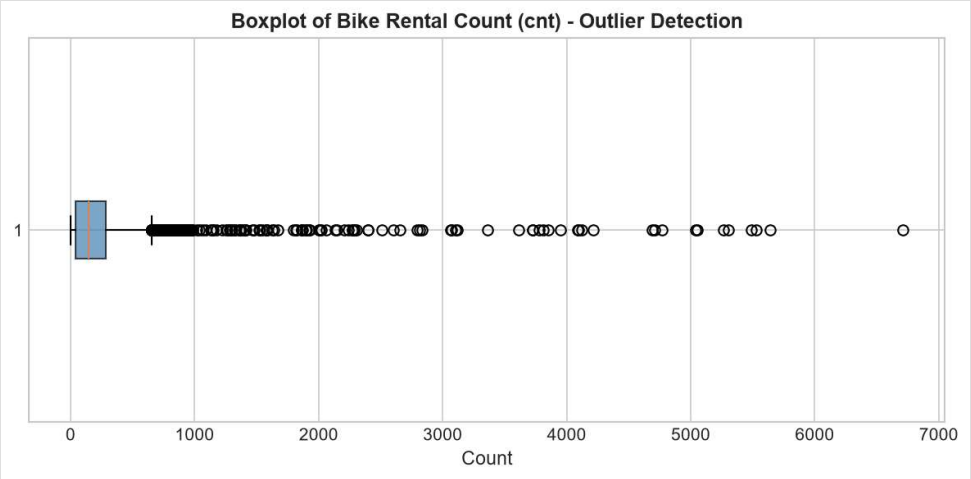


Figure 6. Boxplot of bike rental counts showing outlier points above the upper fence.

7.2 Residual Analysis

Plotting residuals (actual minus predicted) against actual values confirms the outlier issue. At high actual cnt values (above ~600), residuals become large and negative, meaning the model systematically under-predicts these extreme cases. These high-count points are qualitatively different from typical hours and distort the overall MSE evaluation.

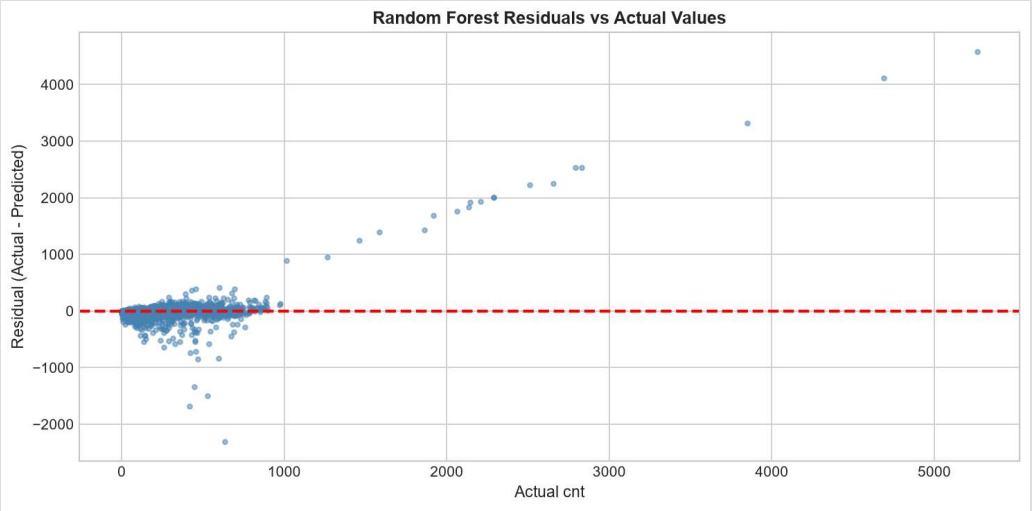


Figure 7. Random Forest residuals vs actual values, showing systematic under-prediction for high-count outliers.

7.3 Removing Outliers

After removing the 577 outlier observations, the cleaned dataset contains 16,802 samples. We re-split and re-preprocess this cleaned dataset for the subsequent optimization steps. Removing outliers provides a fairer comparison of the two algorithms' core predictive performance on typical data points.

8. Hyperparameter Optimization

With the cleaned dataset, we perform hyperparameter tuning for both models to find the best trade-off between accuracy and training time.

8.1 Random Forest Tuning

We vary the number of trees (`n_estimators`) and maximum tree depth (`max_depth`):

n_estimators	max_depth	Train Time (s)	Train MSE	Test MSE
50	None	0.3007	457.9	3,166.0
100	**None**	**0.5217**	**429.0**	**3,123.9**
50	15	0.2014	3,515.2	5,191.5
100	20	0.3379	1,701.5	3,897.3
150	25	0.5753	755.0	3,235.4

The configuration with n_estimators=100 and unlimited max_depth achieves the lowest test MSE of 3,123.9. The full-depth trees allow the model to fit the training data very closely (train MSE of only 429), but this comes at the cost of substantial overfitting. Limiting max_depth reduces overfitting but significantly increases test error, suggesting the model benefits from its full capacity despite the overfitting.

8.2 KNN Tuning

We test different values of k (number of neighbors):

k	Train Time (s)	Train MSE	Test MSE
3	0.0034	2,661.0	5,996.3
5	0.0032	3,716.5	6,015.1
7	0.0036	4,281.2	5,924.2
9	0.0029	4,489.0	5,819.1
11	**0.0027**	**4,645.4**	**5,752.5**
15	0.0034	5,032.0	5,867.7
20	0.0026	5,574.2	6,292.9
25	0.0024	6,085.9	6,728.1

The optimal k value is 11, yielding the lowest test MSE of 5,752.5. Beyond k=11, test MSE starts to increase as the model becomes overly smoothed and begins to underfit. KNN training time remains negligible (under 0.004 seconds) across all k values since "training" simply stores the data in memory.

9. Final Results and Comparison

9.1 Final Optimized Models

Based on the tuning results (lowest test MSE), we select the following final configurations:

- **Random Forest:** n_estimators=100, max_depth=None
- **KNN:** n_neighbors=11

9.2 Performance Summary Table

Metric	Random Forest	KNN
Training Time (seconds)	0.5217	0.0027
Train MSE	429.01	4,645.43
Test MSE	3,123.94	5,752.51
Final Hyperparameters	n_estimators=100, max_depth=None	n_neighbors=11

9.3 Final Predicted vs Actual Plots

After outlier removal and hyperparameter optimization, the predicted vs actual plots show much cleaner alignment with the diagonal. The Random Forest plot shows points tightly clustered along the diagonal line, demonstrating strong predictive performance. The KNN plot still shows wider scatter and the characteristic horizontal banding pattern of KNN regression.

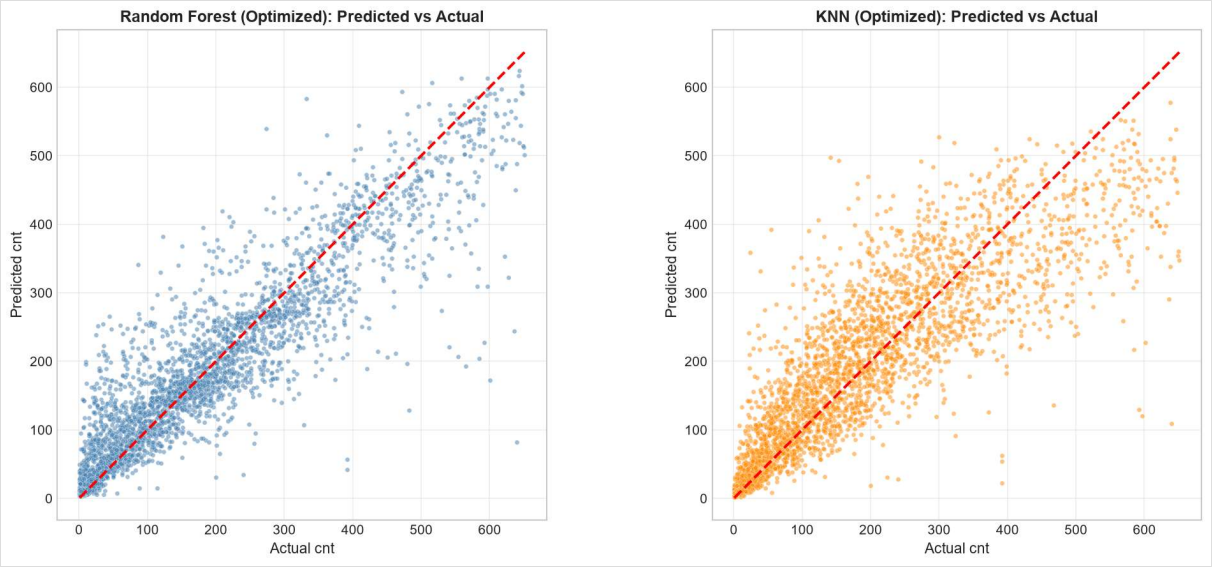


Figure 8. Final predicted vs actual rental counts after outlier removal and optimization. Left: Random Forest. Right: KNN.

9.4 Feature Importance (Random Forest)

The Random Forest model provides feature importance scores. As expected from the exploratory analysis, the hour of day (`hr`) features are collectively the most important predictors, followed by temperature and the year indicator. This aligns with our observation of strong daily commuting patterns and year-over-year growth in bike sharing usage.

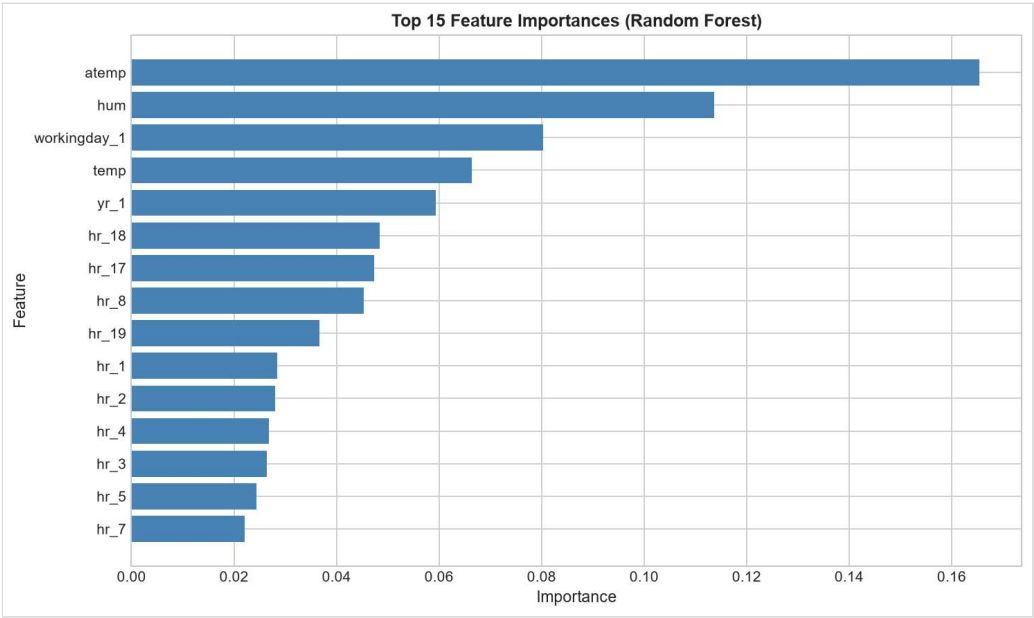


Figure 9. Top 15 most important features as determined by Random Forest feature importances.

10. Discussion and Conclusion

10.1 Performance Comparison

Random Forest significantly outperforms KNN on this bike sharing prediction task. The test MSE of Random Forest (3,123.9) is nearly half that of KNN (5,752.5), meaning Random Forest achieves substantially better predictive accuracy. This performance gap is expected because Random Forest can model complex non-linear relationships and feature interactions — for example, how the effect of temperature varies by season, or how hour-of-day patterns differ between workdays and weekends. KNN, by contrast, struggles in the high-dimensional one-hot encoded feature space where distance metrics become less discriminative (the curse of dimensionality).

10.2 Overfitting Analysis

Random Forest shows much stronger signs of overfitting. This is evident from the very large gap between its training MSE (429.0) and test MSE (3,123.9) — roughly a 7.3x difference. The model fits the training data extremely well but generalizes noticeably worse to unseen data. This is typical behavior for Random Forest with fully grown trees; each individual tree can memorize the training data, and the ensemble retains enough capacity to overfit despite the averaging effect.

KNN shows a much smaller relative gap between train and test MSE (4,645.4 vs 5,752.5, about 1.24x), indicating far less overfitting. However, this is because KNN underfits more fundamentally — it simply cannot capture the complex patterns in the data well, so both training and test errors are high but close to each other. A model that underfits will naturally show less gap between train and test performance.

10.3 Training Time

There is a very large difference in training time. KNN training is essentially instantaneous (0.003 seconds) because it is a lazy learner — no model is built during training, the data is simply stored in memory. All computation happens at prediction time. Random Forest requires building 100 decision trees, taking about 0.5 seconds. While this is a large relative difference (roughly 200x), both models train quickly enough that training time is not a practical concern for this dataset size.

10.4 Effect of Outlier Removal

Removing outliers dramatically improved the apparent quality of both models. Before outlier removal, test MSE values were an order of magnitude higher (39,552 for RF, 44,593 for KNN) because the extreme high-count points dominated the squared error calculation. After removal, test MSE dropped

to 3,124 and 5,753 respectively, and the predicted vs actual plots became much cleaner. This demonstrates how sensitive MSE is to outliers and why data cleaning is an important step.

10.5 Conclusion

For the bike sharing hourly demand prediction problem, **Random Forest is clearly the superior regressor**. It achieves nearly 50% lower test MSE despite exhibiting much more overfitting tendency. The KNN model is extremely fast to train but its prediction accuracy is inadequate for this task, largely due to the high-dimensional categorical feature space degrading the meaningfulness of distance-based similarity.

Key Takeaways:

- Random Forest achieves ~46% lower test MSE than KNN (3,124 vs 5,753)
- Random Forest shows far stronger overfitting (train/test ratio ~7.3x vs ~1.24x for KNN)
- Hour of day and temperature are the most important predictive features
- Outlier removal (IQR method, 3.3% of data removed) drastically reduced MSE values
- KNN training is ~200x faster but Random Forest is still very fast (<1 second)
- Random Forest is recommended for this bike rental prediction task

---

End of Final Project Report